

AWS DeepRacer local training configuration

How to train the AWS DeepRacer vehicle locally, without the AWS console

Jakl Vincent¹, Kosorin Peter², and Procházka Adam³

¹jaklvinc@fit.cvut.cz

²kosorpet@fit.cvut.cz

³prochada@fit.cvut.cz

Abstract

AWS DeepRacer is an innovative platform developed by Amazon Web Services that enables developers to build, train, and deploy intelligent autonomous driving models using reinforcement learning. This article explores how to train the models independently of AWS, using only a local machine. This work builds heavily upon the work done by AWS DeepRacer community.

1 Keywords

AWS, DeepRacer, Autonomous Driving, Reinforcement Learning, Deep Learning

2 Deepracer-for-cloud Overview

AWS DeepRacer is a platform developed by Amazon Web Services that enables developers to build, train, and deploy intelligent autonomous driving models using reinforcement learning. The platform includes a physical car, known as the DeepRacer, equipped with lidar and cameras, as well as a cloud-based simulator for training and evaluation. Developers use reinforcement learning to teach the DeepRacer to navigate a course as quickly and efficiently as possible. The DeepRacer is rewarded for completing the course quickly and penalized for leaving the track or colliding with obstacles. The platform also includes tools for visualizing and analyzing the performance of the DeepRacer model.

2.1 Local Training

For cost and time saving, training on a local machine is preferable. Local training is enabled by the deepracer-for-cloud environment developed by AWS DeepRacer community. The environment is based on AWS RoboMaker and AWS SageMaker containers, supporting both gpu and cpu setups.

2.2 Main Features

Deepracer-for-cloud supports wide set of features for creating models:

- Modes
 - Time Trial
 - Object Avoidance
 - Head-to-Bot
- Training
 - Multiple RoboMaker instances per SageMaker (N:1) to improve training progress
 - Multiple training sessions in parallel - each being (N:1) if hardware supports it - to test out things in parallel
 - Connect multiple nodes together (Swarm-mode only) to combine the powers of multiple computers/instances
- Evaluation
 - Evaluate independently of training
 - Save evaluation run to MP4 file in S3
- Logging
 - Training metrics and trace files are stored to S3
 - Optional integration with AWS CloudWatch
 - Optional exposure of Robomaker internal log-files
- Technology
 - Supports both Docker Swarm (used for connecting multiple nodes together) and Docker Compose (used to support OpenGL)

3 Installation

The package comes with preparation and setup scripts that allow a turn-key setup. First, clone the repository:

```
git clone https://github.com/aws-deepracer-community/deepracer-for-cloud.git
```

Next, ensure that all prerequisites are set up and run the `bin/init.sh` script.

3.1 Environment setup

Local mode runs a minio server that hosts the data in the `docker/volumes` directory. Configure the Minio credentials with:

```
aws configure --profile minio
```

The default configuration will use the `minio` profile to configure MINIO. You can choose any username or password, but the username needs to be at least length 3, and the password at least length 8. Next, configure `system.env` as follows:

- `DR_LOCAL_S3_PROFILE=default`
- `DR_LOCAL_S3_BUCKET=<bucketname>`

Finally, run `dr-update` for configuration to take effect.

3.2 First run

For the first run, the following steps are needed. This creates a training run with all the default values set. First, define your custom files in `custom_files/`, samples can be found in the `defaults` folder:

- `hyperparameters.json`
- `model_metadata.json`
- `reward_function.py`

Upload the files into the bucket with `dr-upload-custom-files`. This will also start minio if required. Finally, start training with `dr-start-training`.

4 Training customization

Training locally allows for a wide degree of training customization compared to the official AWS DeepRacer console.

4.1 Running multiple experiments

It is possible to run multiple experiments on one computer in parallel. This is controlled by `DR_RUN_ID` in `run.env`. It is also important to know that, even though you can run multiple training instances on one local machine, they will all use the same minio container. So not only is it important to change the `DR_RUN_ID` but also `DR_LOCAL_S3_MODEL_PREFIX`, which if left default, can collide with the other, already trained models. Follow the following steps to run multiple training instances:

1. Check if there is any training currently running on the machine using `docker ps`
2. Change the `DR_RUN_ID` in your `run.env` file, so that it does not clash with already running training instances
3. Change the `DR_LOCAL_S3_MODEL_PREFIX` to one that was not yet used in local training
4. Set up custom files and upload them with `dr-upload-custom-files`
5. Start training with `dr-start-training`

4.2 Modifying the reward function

The AWS DeepRacer reward function takes a dictionary object as the input:

```
def reward_function(params) :  
  
    reward = ...  
  
    return float(reward)
```

For a detailed description of the input parameters see the official AWS DeepRacer documentation. AWS DeepRacer supports the following libraries: `math`, `random`, `NumPy`, `SciPy`, and `Shapely`. To use one, add an import statement to import a supported library above your function definition.

4.3 Neural network architecture

Each agent (the vehicle) has a neural network for choosing the appropriate actions. The network is designed in a modular way, separated into three following parts: input embedder, middleware and the output head.

4.3.1 CNN image input embedders

This is the first stage of the network, meant to convert the input into a feature vector representation. It is possible to combine several instances of any of the supported embedders, in order to allow varied combinations of inputs. The type of Input Embedder that AWS DeepRacer uses is Convolutional Neural Network. Conceptually, this portion of the network is what identifies the features of the track, such as the center line, outer lines, the outer wall, off-track areas, and track artwork.

One can modify `/markov/sensors/utils.py` in sagemaker to modify the shape of the convolutional network layers for a given sensor type.

4.3.2 Fully Connected Middleware

The middleware gets the output of the input embedder, and processes it into a different representation domain, before sending it through the output head. The goal of the middleware is to enable processing the combined outputs of several input embedders, and pass them through some extra processing. This, for instance, might include an LSTM or just a plain simple FC layer. Conceptually, this portion of the network is what 'thinks' about what to do with the features identified in the earlier layers. Hopefully, these layers learn to make decisions based on relevant features of the road, such as the center and outer lines, and learn to ignore other features, such as the outer wall and track artwork.

There are three built-in choices for middleware shapes, but one can customize it further if desired.

- `["DEEP_CONVOLUTIONAL_NETWORK"]`
 - This includes a single fully connected layer of 64 nodes.
- `["DEEP_CONVOLUTIONAL_NETWORK_SHALLOW"]`
 - No fully connected layer; the input embedder nodes connect directly to the action nodes
- `["DEEP_CONVOLUTIONAL_NETWORK_DEEP"]`
 - This includes two fully connected layers of 512 nodes each.

If you'd like to customize the code to add more shape choices, modify the code in `/markov/sensors/utils.py` in the SageMaker container.

4.3.3 Output head

The output head is used in order to predict the values required from the network. This area of the network is what determines the action the DeepRacer vehicle takes, specifically the steering and the throttle settings. In a discrete action space, there are a set number of output nodes, each one representing a specific combination of steering and throttle. In a continuous action space, there is a single output node, but it returns two variables, one for steering and one for throttle, each of which can be anywhere inside a given range. Discrete action spaces tend to train faster, but continuous action spaces allow for more nuanced decisions.

4.3.4 Modify the action space

In a discrete action space, each node in the final layer of the neural network represents a combination of steering and throttle. For such, the `model_metadata.json` file will contain a list of those actions. For example, a custom 5-node discrete action space may look like this:

```
"action_space": [
  {
    "steering_angle": -30,
    "speed": 0.5
  },
  {
    "steering_angle": -15,
    "speed": 1.0
  },
  {
    "steering_angle": 0,
    "speed": 1.0
  },
  {
    "steering_angle": 15,
```

```

        "speed": 1.0
    },
    {
        "steering_angle": 30,
        "speed": 0.5
    }
],

```

In a continuous action space, there is a single node in the neural network for speed and steering respectively, but it can return a variable inside a given range. An example may be:

```

"action_space" : {
    "steering_angle" : {
        "high" : 30.0,
        "low" : -30.0
    },
    "speed" : {
        "high" : 2.0,
        "low" : 0.5
    }
}

```

4.3.5 Final network shape

The exact shape of the final network will depend upon the combination of input sensors chosen combined with the middleware selection and the action section. See Table 1 for example network shapes.

5 Training and evaluation log analysis

To run log analysis, run the following command:

```
dr-start-loganalysis
```

This will start jupyter lab on `localhost:8888`. If you are using a server, make sure to use a proper tunnel.

There are already good descriptions for evaluation. However for local training there are a few changes that need to be made in order to make the log analysis work.

5.1 Metrics.ipynb

Metrics notebook is used to check training metrics, and to see whether the training has converged yet. It is mostly used to see the training progression through time.

In this notebook, the following changes are needed.

- In the second code cell, set the following parameters:

```

PREFIX='DR_LOCAL_S3_MODEL_PREFIX'
BUCKET='DR_LOCAL_S3_BUCKET'
PROFILE='DR_LOCAL_S3_PROFILE'

```

Where the DR variables are the same as in your `run.env` and `system.env`

- Comment out the third code cell.
- Set the fourth cell to this:

```

tm = metrics.TrainingMetrics(BUCKET, model_name=PREFIX,
                             profile=PROFILE, s3_endpoint_url='localhost:9000

```

Use tunnelled IP instead of localhost, if you are running the analysis on a server.

This should be enough to run the Metrics.ipynb

5.2 Training analysis.ipynb

This notebook is mainly used to explore if your reward function is set up properly as well as your action space. It is useful for finding unwanted behaviours of the vehicle.

For local training, this notebook also needs some changes:

- In the first code cell add this import statement

```
from deepracer.logs import (AnalysisUtils,  
                             DeepRacerLog,  
                             S3FileHandler)
```

- In the second code cell replace or comment out the following lines:

```
model_logs_root = 'logs/sample-console-logs'  
log = DeepRacerLog(model_logs_root)
```

instead, add these lines:

```
fh = S3FileHandler(bucket="BUCKET", prefix="PREFIX",  
                   profile="PROFILE", s3_endpoint_url="localhost:9000")  
log = DeepRacerLog(filehandler=fh)
```

Where the variables BUCKET, PREFIX, PROFILE are the same as in Metrics.ipynb. The same applies to the s3_endpoint_url.

- Change the track_name to the track that you have been training on.

After these changes, the notebook should be usable.

Table 1. Example network shapes.

DEEP_CONVOLUTIONAL_NETWORK	DEEP_CONVOLUTIONAL_NETWORK_SHALLOW	DEEP_CONVOLUTIONAL_DEEP
Conv2d(32,5,2)	Conv2d(32,8,4)	Conv2d(32,8,4)
Conv2d(32,3,1)	Conv2d(64,4,2)	Conv2d(32,4,2)
Conv2d(64,3,2))	Conv2d(64,3,1)	Conv2d(64,4,2)
Conv2d(64,3,1)		Conv2d(64,3,1)
dense_layer(64)		dense_layer(512)
		dense_layer(512)